

# Pràctica 2

---

## Pràctica A: Interrupció per GPIO

```
#include <Arduino.h>
struct Button {
    const uint8_t PIN;
    uint32_t numberKeyPresses;
    bool pressed;
};

Button button1 = {18, 0, false};

void IRAM_ATTR isr() {
    button1.numberKeyPresses += 1;
    button1.pressed = true;
}

void setup() {
    Serial.begin(115200);
    pinMode(button1.PIN, INPUT_PULLUP);
    attachInterrupt(button1.PIN, isr, FALLING);
}

void loop() {
    if (button1.pressed) {
        Serial.printf("Button 1 has been pressed %u times\n",
            button1.numberKeyPresses);
        button1.pressed = false;
    }

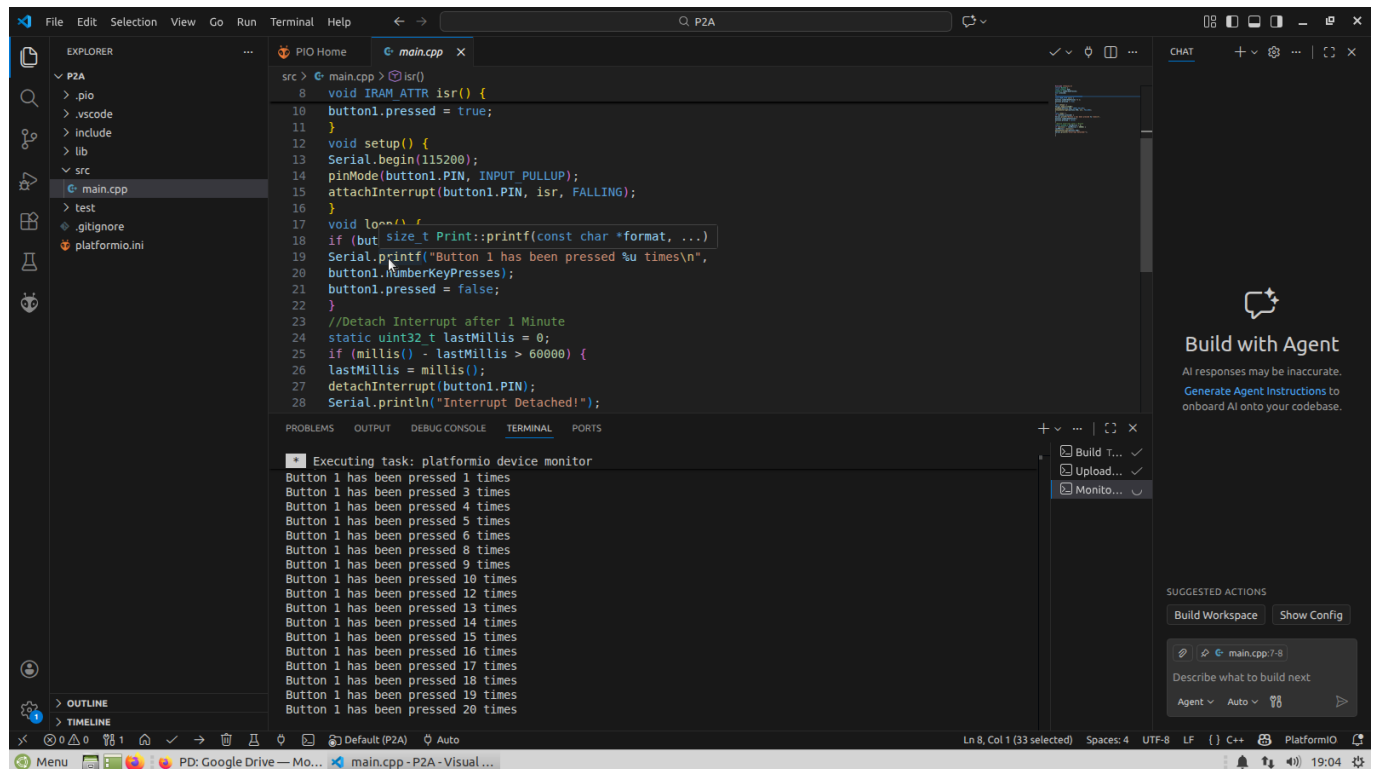
    //Detach Interrupt after 1 Minute
    static uint32_t lastMillis = 0;
    if (millis() - lastMillis > 60000) {
        lastMillis = millis();
        detachInterrupt(button1.PIN);
        Serial.println("Interrupt Detached!");
    }
}
```

En aquest codi s'implementa un sistema d'interrupcions per a comptar quantes vegades es pressiona un botó connectat al pin 18. En lloc de revisar constantment l'estat del botó en el bucle principal, el programa utilitza una funció especial anomenada `isr ()` (Interrupt Service Routine) que es dispara automàticament cada vegada que el pin detecta una caiguda de voltatge (`FALLING`), incrementant un comptador i activant una bandera d'avís.

En el `loop()`, el processador verifica si la bandera de pressionat és veritable per a imprimir el comptatge total pel monitor seriï. A més, el codi inclou un temporitzador lògic que, després de passar 60 segons des

de l'inici, desactiva la interrupció mitjançant `detachInterrupt()`. Això deté el control del botó, fent que el programa deixi de reaccionar a les pulsacions després d'aquest minut d'execució.

## Sortida de la impressió al port sèrie



## Practica B: Interrupció per timer

```
#include <Arduino.h>
volatile int interruptCounter;
int totalInterruptCounter;
hw_timer_t * timer = NULL;
portMUX_TYPE timerMux = portMUX_INITIALIZER_UNLOCKED;

void IRAM_ATTR onTimer() {
  portENTER_CRITICAL_ISR(&timerMux);
  interruptCounter++;
  portEXIT_CRITICAL_ISR(&timerMux);
}

void setup() {
  Serial.begin(115200);
  timer = timerBegin(0, 80, true);
  timerAttachInterrupt(timer, &onTimer, true);
  timerAlarmWrite(timer, 1000000, true);
  timerAlarmEnable(timer);
}

void loop() {
  if (interruptCounter > 0) {
    portENTER_CRITICAL(&timerMux);
    interruptCounter--;
```

```

portEXIT_CRITICAL(&timerMux);
totalInterruptCounter++;
Serial.print("An interrupt as occurred. Total number: ");
Serial.println(totalInterruptCounter);
}
}

```

En aquest programa, es configura un temporitzador intern que funciona com una alarma precisa de fons. Cada vegada que passa exactament un segon, el temporitzador llança una "interrupció", cosa que significa que el processador pausa momentàniament el que està fent per a executar la funció `onTimer`. En aquesta funció, simplement se suma un a un comptador ràpid anomenat `interruptCounter`.

En el bloc principal `loop()`, el programa revisa constantment si el comptador ha augmentat. Si detecta que va ocórrer una interrupció, descompta aquest avís, incrementa un comptador total i mostra un missatge en el monitor seriï confirmant que ha passat un segon més. L'ús de comandos com `portENTER_CRITICAL` assegura que la comunicació entre l'alarma i el programa principal sigui segura i no es perdin dades per intentar accedir al mateix número al mateix temps.

### Sortida de la impressió al port sèrie

```

P2B > src > main.cpp > setup()
4  hw_timer_t * timer = NULL;
5  portMUX_TYPE timerMux = portMUX_INITIALIZER_UNLOCKED;
6  void IRAM_ATTR onTimer() {
7  portENTER_CRITICAL_ISR(&timerMux);
8  interruptCounter++;
9  portEXIT_CRITICAL_ISR(&timerMux);
10 }
11 void setup() {
12 Serial.begin(115200);
13 timer = timerBegin(0, 80, true);
14 timerAttachInterrupt(timer, &onTimer, true);
15 timerAlarmWrite(timer, 1000000, true);
16 timerAlarmEnable(timer);
17 }
18 void loop() {
19 if (interruptCounter > 0) {
20 portENTER_CRITICAL(&timerMux);
21 interruptCounter--;
22 portEXIT_CRITICAL(&timerMux);
23 totalInterruptCounter++;

```

Executing task in folder P2B: platformio device monitor

```

An interrupt as occurred. Total number: 67
An interrupt as occurred. Total number: 68
An interrupt as occurred. Total number: 69
An interrupt as occurred. Total number: 70
An interrupt as occurred. Total number: 71
An interrupt as occurred. Total number: 72
An interrupt as occurred. Total number: 73
An interrupt as occurred. Total number: 74
An interrupt as occurred. Total number: 75
An interrupt as occurred. Total number: 76
An interrupt as occurred. Total number: 77
An interrupt as occurred. Total number: 78
An interrupt as occurred. Total number: 79
An interrupt as occurred. Total number: 80
An interrupt as occurred. Total number: 81
An interrupt as occurred. Total number: 82
An interrupt as occurred. Total number: 83
An interrupt as occurred. Total number: 84

```

## Pràctica 2: Part complementaria

### Enunciat

Teneis que hacer que chatgpt os genere un codigo para esp32 arduino

Sugerencia de prompt: Quiero hacer un programa para arduino esp32 en el entorno de platformio ; que controle un led y dos pulsadores utilizando interrupciones de un timer de forma que el led parpadee a una frecuencia inicial y si pulsamos a un pulsador la frecuencia de parpadeo suba y si pulsamos a otro pulsador dicha frecuencia baje ; garantiza de que el pulsador los pulsadores que se deben de leer en el timer se filtran para evitar rebotes

Teneis que describir como funciona el codigo generado por chat gpt y probarlo.

### Codi

```
#include <Arduino.h>

/* =====
PINES
===== */
#define LED_PIN      2
#define BTN_UP_PIN   47
#define BTN_DOWN_PIN 48

/* =====
PARÁMETROS
===== */
#define TIMER_INTERVAL_US 1000
#define DEBOUNCE_TIME_MS  20

#define FREQ_MIN 1
#define FREQ_MAX 20

/* =====
VARIABLES
===== */

hw_timer_t *timer = NULL;
portMUX_TYPE timerMux = portMUX_INITIALIZER_UNLOCKED;

volatile bool tick_1ms = false;

uint32_t led_counter = 0;
bool led_state = false;
```

```
uint8_t led_frequency = 2;
uint8_t last_printed_frequency = 0;

/* Debounce */
uint8_t btn_up_state = HIGH;
uint8_t btn_down_state = HIGH;

uint8_t btn_up_last = HIGH;
uint8_t btn_down_last = HIGH;

uint16_t btn_up_debounce = 0;
uint16_t btn_down_debounce = 0;

/* =====
   ISR
   ===== */
void IRAM_ATTR onTimer()
{
    portENTER_CRITICAL_ISR(&timerMux);
    tick_1ms = true;
    portEXIT_CRITICAL_ISR(&timerMux);
}

/* =====
   SETUP
   ===== */
void setup()
{
    Serial.begin(115200);
    delay(1000);

    Serial.println("Sistema iniciado");
    Serial.println("Frecuencia inicial: 2 Hz");

    pinMode(LED_PIN, OUTPUT);
    pinMode(BTN_UP_PIN, INPUT_PULLUP);
    pinMode(BTN_DOWN_PIN, INPUT_PULLUP);

    timer = timerBegin(0, 80, true);
    timerAttachInterrupt(timer, &onTimer, true);
    timerAlarmWrite(timer, TIMER_INTERVAL_US, true);
    timerAlarmEnable(timer);
}

/* =====
   LOOP
   ===== */
void loop()
{
    if (tick_1ms)
    {
        portENTER_CRITICAL(&timerMux);
        tick_1ms = false;
        portEXIT_CRITICAL(&timerMux);
    }
}
```

```
/* ===== CONTROL LED ===== */
uint32_t half_period = 500 / led_frequency;

if (++led_counter >= half_period)
{
    led_state = !led_state;
    digitalWrite(LED_PIN, led_state);
    led_counter = 0;
}

/* ===== LECTURA BOTONES ===== */
uint8_t up = digitalRead(BTN_UP_PIN);
uint8_t down = digitalRead(BTN_DOWN_PIN);

// SUBIR
if (up != btn_up_last)
    btn_up_debounce = 0;
else if (btn_up_debounce < DEBOUNCE_TIME_MS)
    btn_up_debounce++;
else if (up != btn_up_state)
{
    btn_up_state = up;
    if (btn_up_state == LOW && led_frequency < FREQ_MAX)
        led_frequency++;
}
btn_up_last = up;

// BAJAR
if (down != btn_down_last)
    btn_down_debounce = 0;
else if (btn_down_debounce < DEBOUNCE_TIME_MS)
    btn_down_debounce++;
else if (down != btn_down_state)
{
    btn_down_state = down;
    if (btn_down_state == LOW && led_frequency > FREQ_MIN)
        led_frequency--;
}
btn_down_last = down;
}

/* ===== IMPRIMIR SI CAMBIA FRECUENCIA ===== */
if (led_frequency != last_printed_frequency)
{
    Serial.print("Frecuencia actual: ");
    Serial.print(led_frequency);
    Serial.println(" Hz");

    last_printed_frequency = led_frequency;
}
}
```